Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Systems@ETH Zürich

Cloud Computing Architecture

Semester project report

Group 54
Aaron Achermann - 20-940-805
Indro Born - 21-936-208
Matteo Pelossi – 22-952-444

Systems Group
Department of Computer Science
ETH Zurich
May 13, 2026

Instructions

- **Please do not modify the template!** Except for adding your solutions in the labeled places, and inputting your group number, names and legi-NR on the title page.

  Divergence from the template can lead to subtraction of points on graded parts of the project.

- Parts 1 and 2 of the project are **ungraded**. They will help you analyze the behavior of the applications you will have to run on parts 3 and 4 (graded), for which you will have to design a scheduling policy **based on the information** you will gather on **parts 1 and 2**.

- Be conservative with your credits! Parts 3 and 4 might need extra credits for experimentation. Parts 1 and 2 should cost you approximately **15 USD**.

- **Use of AI Tools**: The use of AI tools must adhere to ETH regulations. **You take responsibility for the content you submit**. You must disclose any use of GenAI and other AI tools in your work and avoid sharing copyrighted, private, or confidential information with commercial AI platforms. Failure to comply with these guidelines may result in disciplinary action.

Part 3 [68 points]

1. **[34 points]** Describe and analyze your hand-crafted policy.

   (a) **[17 points]** With your scheduling policy, run the entire workflow **3 separate times**. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached, running with a steady client load of 30K QPS. For each batch application, compute the mean and standard deviation of the execution time [1] across three runs. Also, compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Finally, compute the SLO violation ratio for memcached for the three runs; the number of data points with 95th percentile latency $>$ 1ms, as a fraction of the total number of data points. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

job name	mean time [s]	std [s]
barnes		
blackscholes		
canneal		
freqmine		
radix		
streamcluster		
vips		
total time		

   Answer:

   Create 3 bar plots (one for each run) of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Using the augmented version of mcperf, you get two additional columns in the output: `ts_start` and `ts_end`. Use them to determine the width of each bar in the bar plot, while the height should represent the p95 latency. Align the x axis so that $x = 0$ coincides with the starting time of the first container. For each core in each machine show what job it is running using the colors proposed in this template (you can find them in `main.tex`). For example, draw a bar in the color of vips for core x from when vips is started to when it ends if vips ran on core x.

   Plots:

   (b) **[17 points]** Describe and justify the "optimal" scheduling policy you have designed.

   - Which node does memcached run on? Why?
     Answer:

   - Which node does each of the 7 batch jobs run on? Why?
     Answer:
     - barnes:

[1] Here, you should only consider the runtime, excluding time spans during which the container is paused.

  - blackscholes:
  - canneal:
  - freqmine:
  - radix:
  - streamcluster:
  - vips:

- Which jobs run concurrently / are colocated? Why?
  Answer:

- In which order did you run the 7 batch jobs? Why?
  Order (to be sorted): barnes, blackscholes, canneal, freqmine, radix, streamcluster, vips
  Why:

- How many threads have you used for each of the 7 batch jobs? Why?
  Answer:
  - barnes:
  - blackscholes:
  - canneal:
  - freqmine:
  - radix:
  - streamcluster:
  - vips:

- Which files did you modify or add and in what way? Which Kubernetes features did you use?
  Answer:

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above). Describe how your policy (and its performance) compares to another policy that you experimented with (in particular to the second-best policy that you designed).
  Answer:

2. **[34 points]** Describe and analyze the AI-generated policy. [TODO: Xiaozhe]

   (a) **[17 points] Report the performance**:

   - Report the mean time of each batch job, as well as the makespan of all jobs for the AI-generated policy. Also, report the SLO violation ratio for memcached for the AI-generated policy. *Note: If the LLM failed to produce a valid solution after 3+ attempts, provide the error logs and your best-performing manual tweak for partial credit.*

job name	mean time [s] (Baseline)	mean time [s] (AI)
barnes		
blackscholes		
canneal		
freqmine		
radix		
streamcluster		
vips		
total time		

- Create 3 bar plots (one for each run), for the final AI-generated policy, of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Use the same method as described in the previous question to create the bar plots.
  Answer:
- Create two line plots (one for the p95 latency and one for SLO violations) showing policy performance over iterations.
  Answer:

(b) **[17 points]** Analysis of the Evolutionary Process: Describe the process of how you used OpenEvolve to design the AI-generated policy. For example, you can include the following details in your answer (You are not limited to these questions. You can include any other details you find relevant and interesting). Your response will be graded based on the thoroughness of the exploration and clarity in explanation:

- How many iterations did you run through OpenEvolve with the LLMs to get to the final policy? Which LLM(s) did you use?
- How do you measure success? For example, how do you design the 'combined_score' or the success metric, given that there are two objectives (makespan and SLO violations)?
- Describe the initial policy you fed to the LLM(s) and why you chose this as the initial policy.
- Describe how the policy changes during the process, and how it impacted the performance of the policy (e.g., tail latency; SLO violations).
- Explain why did this AI-generated policy perform better (or worse) than the baseline?
- Identify one "hallucination" or logical flaw the model produced during the process and how you identified it.

Answer:

Please attach your modified/added YAML files, run scripts, OpenEvolve scripts, experiment outputs and the report as a zip file. You can find more detailed instructions about the submission in the project description file.

Important: The search space of all possible policies is exponential and you do not have enough credits to run all of them. We do not ask you to find the policy that minimizes the total running time, but rather to design a policy that has a reasonable running time, does not violate the SLO, and takes into account the characteristics of the first two parts of the project.

Part 4 [75 points]

1. **[19 points]** Use the following `mcperf` command to vary QPS from 5K to 125K in order to answer the following questions:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP  \
           --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 2 \
           --scan 5000:125000:10000
```

   a) [8 points] How does memcached performance vary with the number of threads (T) and number of cores (C) allocated to the job? In a single graph, plot the 95th percentile latency (y-axis) vs. achieved QPS (x-axis) of memcached (running alone, with no other jobs collocated on the server) for the following configurations (one line each):

   - Memcached with T=1 thread, C=1 core
   - Memcached with T=1 thread, C=2 cores
   - Memcached with T=1 thread, C=3 cores
   - Memcached with T=2 threads, C=1 core
   - Memcached with T=2 threads, C=2 cores
   - Memcached with T=2 threads, C=3 cores
   - Memcached with T=3 thread, C=1 cores
   - Memcached with T=3 thread, C=2 cores
   - Memcached with T=3 threads, C=3 cores

   Label the axes in your plot. State how many runs you averaged across (we recommend three runs) and include error bars. The readability of your plot will be part of your grade.

   Plots:

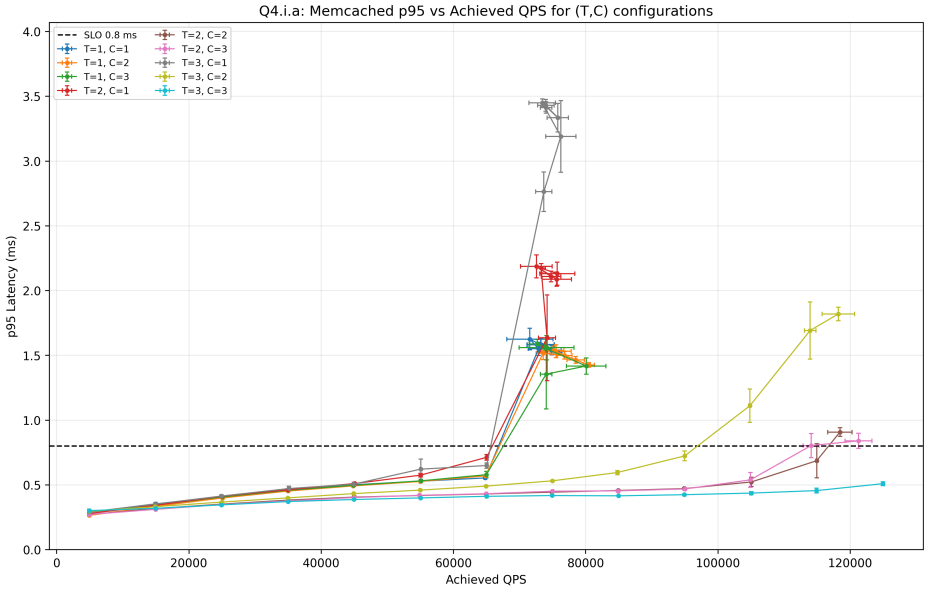

Figure 1: Memcached p95 latency vs. achieved QPS for different thread/core configurations.

What do you conclude from the results in your plot? Summarize in 2-3 brief sentences how memcached performance varies with the number of threads and cores and how this relates to the target QPS.

Summary:
Figure 1 shows that memcached performance depends on both the number of software threads and the number of CPU cores assigned to the process. Configurations with only one core become unstable beyond roughly 65k–75k QPS and violate the 0.8 ms p95 SLO. Increasing the number of cores without increasing the number of memcached threads provides little benefit, as shown by the similar behavior of T=1 with C=1, C=2, and C=3. Conversely, increasing the number of threads without providing enough cores leads to oversubscription and worsens tail latency, as observed for T=2,C=1 and T=3,C=1. The best configuration is T=3,C=3, which remains below the SLO even at the highest tested load. Therefore, our controller should reserve fewer cores for memcached at low load to maximize batch progress, but switch to T=2,C=2 or T=2,C=3 at medium/high load and to T=3,C=3 during high-load periods or when p95 approaches the SLO.

b) [2 points] To support the highest load in the trace (around 125K QPS) without violating the 0.8ms latency SLO, how many memcached threads (T) and CPU cores (C) will you need?

Answer: To support the highest load in the trace, around 125K QPS, without violating the 0.8 ms p95 latency SLO, we need $T = 3$ memcached threads and $C = 3$ CPU cores. This configuration is the only one that remains below the SLO at the highest tested loads.

c) [1 point] Assume you can change the number of cores allocated to memcached dynamically as the QPS varies from 5K to 125K, but the number of threads is fixed when you launch the memcached job. How many memcached threads (T) do you propose to use to guarantee the 0.8ms 95th percentile latency SLO while the load varies between 5K to 125K QPS?

Answer: We propose to use $T = 3$ memcached threads. Since the number of threads is fixed at launch time, it must be chosen to handle the highest possible load. With $T = 3$, the controller can dynamically vary the number of allocated cores between $C = 1$, $C = 2$, and $C = 3$ as the QPS changes, while still being able to meet the 0.8 ms p95 latency SLO at high load.

d) [8 points] Run memcached with the number of threads T that you proposed in (c) and measure performance with $C = 1$, $C = 2$ and $C = 3$. Use the aforementioned `mcperf` command to sweep QPS from 5K to 125K.

Measure the CPU utilization on the memcached server at each load time step. Plot the performance of memcached using 1-core ($C = 1$), 2 cores ($C = 2$) and 3 ($C = 3$) in **three separate graphs**, for $C = 1$, $C = 2$ and $C = 3$, respectively. In each graph, plot achieved QPS on the x-axis, ranging from 0 to 125K. In each graph, use two y-axes. Plot the 95th percentile latency on the left y-axis. Draw a dotted horizontal line at the 0.8ms latency SLO. Plot the CPU utilization (ranging from 0% to 100% for $C = 1$, 200% for $C = 2$ and 300% for $C = 3$) on the right y-axis. For simplicity, we do not require error bars for these plots.

Plots:

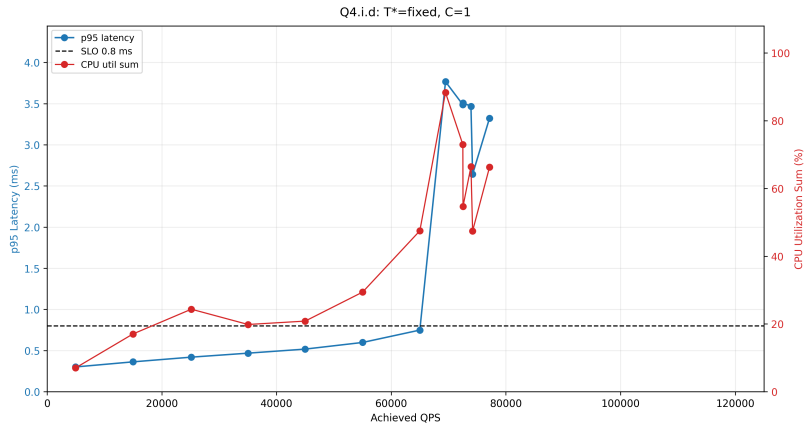

Figure 2: Memcached p95 latency and CPU utilization versus achieved QPS with fixed $T = 3$ and $C = 1$.

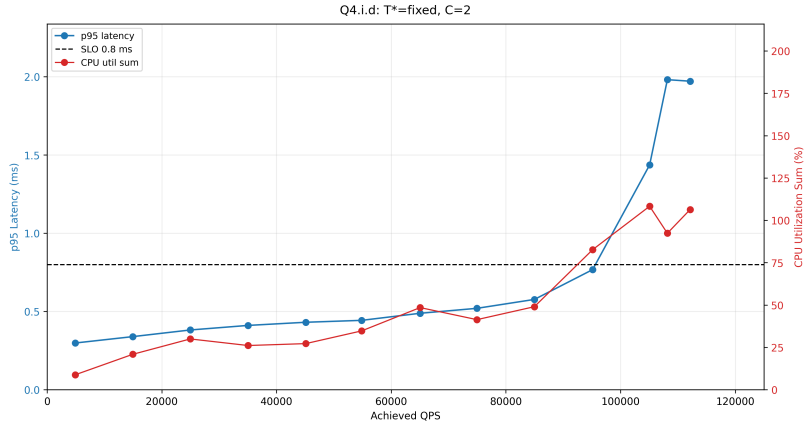

Figure 3: Memcached p95 latency and CPU utilization versus achieved QPS with fixed $T = 3$ and $C = 2$.

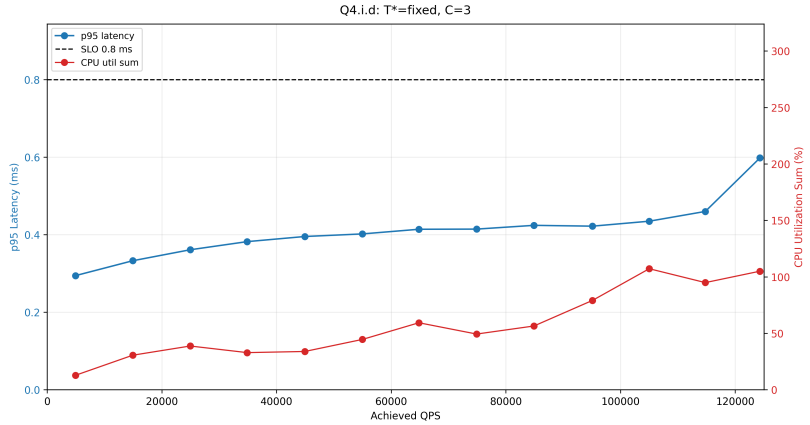

Figure 4: Memcached p95 latency and CPU utilization versus achieved QPS with fixed $T = 3$ and $C = 3$.

With $T = 3$ fixed, the results show that increasing the number of cores allocated to memcached significantly improves its ability to sustain higher QPS while respecting the 0.8 ms p95 latency SLO. With $C = 1$, memcached quickly saturates and violates the SLO once the achieved QPS reaches roughly 70K. With $C = 2$, the SLO is respected for a larger range, but the p95 latency exceeds 0.8 ms at high load. With $C = 3$, memcached remains below the SLO across the full tested range up to approximately 125K QPS.

2. [**17 points**] You are now given a dynamic load trace for memcached, which varies QPS randomly between 5K and 110K in 15 second time intervals. Use the following command to run this trace:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
           --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
           --qps_interval 15 --qps_min 5000 --qps_max 110000
```

Note that you can also specify a random seed in this command using the `--qps_seed` flag.

For this and the next questions, feel free to reduce the mcperf measurement duration (`-t` parameter, now fixed to 30 minutes) as long as you have at the end at least 1 minute of memcached running alone.

Design and implement a controller to schedule memcached and the benchmarks (batch jobs) on the 4-core VM. The goal of your scheduling policy is to successfully complete all batch jobs as soon as possible without violating the 0.8ms 95th percentile latency for memcached. **Your controller should not assume prior knowledge of the dynamic load trace. You should design your policy to work well regardless of the random seed.** The batch jobs need to use the native dataset, i.e., provide the option `-i native` when running them. Also make sure to check that all the batch jobs complete successfully and do not crash. Note that batch jobs may fail if given insufficient resources.

Describe how you designed and implemented your scheduling policy. Include the source code of your controller in the zip file you submit.

- Brief overview of the scheduling policy (max 10 lines):

  Answer: Our controller samples memcached CPU usage every 1 s and dynamically changes the number of cores assigned to memcached between 1 and 3. Memcached is pinned to the low-numbered cores, while Docker batch jobs are restricted to the remaining cores using cpuset updates. The policy prioritizes SLO protection: memcached cores are increased immediately under high per-core CPU load, while cores are released only after sustained low load. To reduce makespan, the final policy uses an opportunistic overlap mode: when memcached needs only one core, the two long critical-path jobs, streamcluster and freqmine, are allowed to run concurrently with a companion job. When memcached needs two or three cores, the policy falls back to a more conservative schedule. The controller does not use future QPS values; it reacts only to runtime CPU signals.
- How do you decide how many cores to dynamically assign to memcached? Why?

Answer: Memcached uses a fixed thread count of $T^* = 3$ and dynamically receives $C_{mem} \in \{1, 2, 3\}$. The controller starts with $C_{mem} = 2$. Every second it computes memcached CPU utilization per assigned core. If utilization exceeds 85% per core, C_{mem} is increased immediately, up to 3 cores, to protect the 0.8 ms p95 latency SLO. If utilization stays below 55% per core for 20 seconds, C_{mem} is reduced by one, down to 1 core, to give more capacity back to batch jobs. This hysteresis avoids oscillations while still reacting quickly to high QPS.

- How do you decide how many cores to assign each batch job? Why?

  Answer: All batch jobs run only on cores not currently used by memcached. If $C_{mem} = 1$, batch jobs can use cores $[1, 2, 3]$; if $C_{mem} = 2$, they use $[2, 3]$; if $C_{mem} = 3$, only core $[3]$ is available. The final policy uses these cores opportunistically: at low memcached load ($C_{mem} = 1$), it overlaps the two long jobs to shorten the critical path; at medium/high memcached load, it schedules more conservatively to protect the SLO.

  - barnes: short/risky companion job; usually 1 core when collocated, more only when long jobs are done.
  - blackscholes: short and relatively safe; 1 core when collocated, up to 2 cores due to weak scaling beyond that.
  - canneal: memory-bandwidth style job; used as a companion at low/medium memcached load, but not launched while $C_{mem} = 3$.
  - freqmine: long critical-path job; overlapped with streamcluster when memcached has enough headroom, but paused first under high memcached pressure.
  - radix: safest short job; can run even when memcached uses 3 cores.
  - streamcluster: long critical-path job; started early and overlapped with freqmine during low-load periods.
  - vips: short but CPU/L1i-risky; scheduled after safer short jobs and may be paused under high load.

- How many threads do you use for each of the batch job? Why? Threads are usually set equal to the number of assigned cores, capped by the per-job maximum. In the opportunistic low-load mode, streamcluster and freqmine use at least 2 software threads even when each receives one core; this kept both long jobs making progress and reduced makespan in our experiments.

  - barnes: $\min(\text{assigned cores}, 4)$
  - blackscholes: $\min(\text{assigned cores}, 2)$
  - canneal: $\min(\text{assigned cores}, 2)$
  - freqmine: $\min(\max(\text{assigned cores}, 2), 4)$ in opportunistic mode
  - radix: $\min(\text{assigned cores}, 4)$
  - streamcluster: $\min(\max(\text{assigned cores}, 2), 4)$ in opportunistic mode
  - vips: $\min(\text{assigned cores}, 4)$

- Which jobs run concurrently / are collocated and on which cores? Why?

  Answer: Memcached always uses the low-numbered cores $[0..C_{mem} - 1]$. Batch jobs use only the remaining high-numbered cores. When $C_{mem} = 1$, the controller can use cores 1–3 and may collocate streamcluster, freqmine, and one companion job. When $C_{mem} = 2$, it usually runs one long job plus one companion job. When $C_{mem} = 3$, only one safe/useful batch job runs on the remaining core, preferring radix or blackscholes.

This keeps risky interference away from memcached during high load while exploiting low-load periods to shorten the batch critical path.

- In which order did you run the batch jobs? Why?

  Order (to be sorted): streamcluster, freqmine, canneal, radix, blackscholes, barnes, vips

  Why: The policy prioritizes streamcluster and freqmine because they are the longest critical-path jobs, so delaying them increases makespan. canneal is started as a companion when enough memcached headroom exists, but it is blocked when memcached needs 3 cores. Among short jobs, radix and blackscholes are scheduled early because they are safer for memcached, while barnes and vips are more CPU/L1i-risky and are therefore scheduled later or paused under pressure.

- How does your policy differ from the policy in Part 3? Why?

  Answer: Part 3 used a static schedule and fixed resource assignment for a fixed-load scenario. Part 4 uses a dynamic QPS trace on one 4-core VM, so a static assignment would either waste cores at low QPS or violate the memcached SLO at high QPS. The Part 4 controller therefore continuously adjusts memcached cores, updates Docker cpusets for batch jobs, and uses pause/unpause as a recovery mechanism. In addition, unlike Part 3, it opportunistically overlaps the long batch jobs only when memcached has enough headroom, and falls back to a conservative schedule when memcached load increases.

- How did you implement your policy? e.g., docker cpu-set updates, taskset updates for memcached, pausing/unpausing containers, etc.

  Answer: The controller runs on the memcache-server VM. Memcached is controlled with `taskset` to update its CPU affinity. Batch jobs are launched as Docker containers with the native input dataset and fixed cpusets. While running, the controller uses Docker cpuset updates to move containers to the currently available batch cores. If memcached per-core CPU exceeds the recovery threshold, the controller pauses the riskiest running container; it unpauses paused containers once memcached utilization falls below the recovery threshold and the job is still part of the desired schedule. All start, end, pause, unpause, and core-update events are logged.

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above). Describe how your policy (and its performance) compares to another policy that you experimented with (in particular to the second-best policy that you designed).

  Answer: The main trade-off is makespan versus memcached SLO protection. The original conservative policy completed Q4.iii in 1208.52 ± 33.00 s with no SLO violations, but it left too much critical-path work serialized. We therefore tested a more aggressive policy that always overlapped the two long jobs; it reduced makespan in some runs but produced SLO violations above the 3% target. The final opportunistic-overlap policy keeps this overlap only when memcached has been reduced to one core, and falls back to the conservative schedule when memcached needs more protection. With this design, Q4.iii improved to 968.19 ± 23.20 s while keeping the SLO violation ratio at 0.0% in all three runs.

3. **[23 points]** Run the following `mcperf` memcached dynamic load trace:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
           --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
           --qps_interval 15 --qps_min 5000 --qps_max 110000 \
           --qps_seed 2345
```

Measure memcached and batch job performance when using your scheduling policy to launch workloads and dynamically adjust container resource allocations. Run this workflow 3 separate times. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached. For each batch application, compute the mean and standard deviation of the execution time [2] across three runs. Compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Also, compute the SLO violation ratio for memcached for each of the three runs; the number of data points with 95th percentile latency $>$ 0.8ms, as a fraction of the total number of datapoints. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

job name	mean time [s]	std [s]
barnes	190.75	0.53
blackscholes	118.64	0.48
canneal	289.54	1.32
freqmine	419.88	23.70
radix	44.99	0.59
streamcluster	543.76	7.14
vips	94.21	0.37
total time	968.19	23.20

Answer: We ran the dynamic mcperf trace with $qps\_interval = 15$ and $qps\_seed = 2345$ three times using our dynamic controller. All seven batch jobs completed successfully in every run. The reported job runtimes exclude time spent paused. The mean makespan over the three runs was 968.19 s with a standard deviation of 23.20 s. The memcached SLO violation ratio was 0.0% in all three runs: run 1 = 0.0%, run 2 = 0.0%, run 3 = 0.0%. Therefore, the controller met the 0.8 ms p95 latency SLO while completing all batch jobs.

Include three plots: one for each of the three runs with the following information: The x-axis should be the time from execution start to end. Use three subplots that share the same x-axis. The first should clearly indicate for each core what job it is running at all time (e.g. using colored boxes). Use the colors proposed in this template (you can find them in `main.text`). If you pause/unpause jobs as part of your scheduling policy this should also be clearly visible. The second subplot should include two y-axis where the left one shows the current QPS and the right one should include the achieved p95 latency. Finally, the third subplot should plot the cpu utilization of each core separately. The readability of your plot will be part of awarded points.

[2] Here, you should only consider the runtime, excluding time spans during which the container is paused.

Plots:

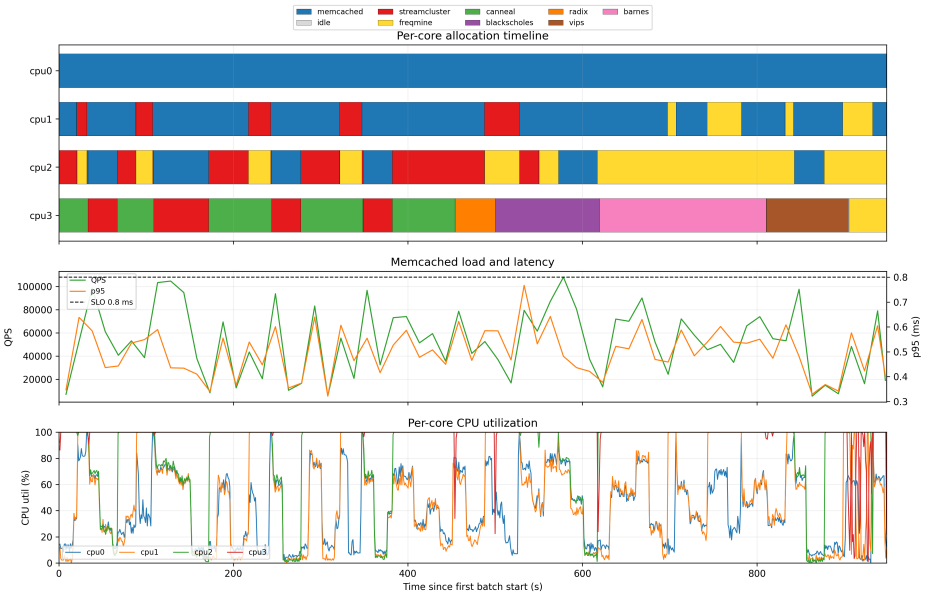

Figure 5: Q4.iii run 1.

Figure 6: Q4.iii run 2.

Figure 7: Q4.iii run 3.

4. **[16 points]** Repeat Part 4 Question 3 with a modified `mcperf` dynamic load trace with a 5 second time interval (`qps_interval`) instead of 15 second time interval. Use the following command:

```
$ ./mcperf -s INTERNAL_MEMCACHED_IP --loadonly
$ ./mcperf -s INTERNAL_MEMCACHED_IP -a INTERNAL_AGENT_IP \
           --noload -T 8 -C 8 -D 4 -Q 1000 -c 8 -t 1800 \
           --qps_interval 5 --qps_min 5000 --qps_max 110000 \
           --qps_seed 2345
```

You do not need to include the plots or table from Question 3 for the 5-second interval. Instead, summarize in 2-3 sentences how your policy performs with the smaller time interval (i.e., higher load variability) compared to the original load trace in Question 3.

Summary: With *qps_interval* = 5, the load changes more frequently than in Question 3, so the controller has less time to react and keeps or returns CPU capacity to memcached more aggressively. This increases pressure on the scheduler compared with the 15 s trace, but the controller still keeps memcached below the required 3% SLO violation threshold. In the 5 s sweep run, the SLO violation ratio was 0.76%, compared with 0.0% average for the original 15 s trace.

What is the SLO violation ratio for memcached (i.e., the number of datapoints with 95th percentile latency > 0.8ms, as a fraction of the total number of datapoints) with the 5-second time interval trace? The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

Answer: The SLO violation ratio for the 5-second interval trace was 0.00756, i.e. 0.76%.

What is the smallest `qps_interval` you can use in the load trace that allows your controller to respond fast enough to keep the memcached SLO violation ratio under 3%?

Answer: The smallest tested `qps_interval` that kept the memcached SLO violation ratio below 3% was 1 second.

What is the reasoning behind this specific value? Explain which features of your controller affect the smallest `qps_interval` interval you proposed.

Answer: We tested intervals $15, 10, 7, 5, 3, 2, 1$ seconds. The controller still stayed below the 3% SLO violation threshold at 1 second: the sweep run had 0.95% violations. This works because the controller samples CPU utilization every second, immediately increases memcached's CPU allocation when pressure is high, and pauses/unpauses batch jobs when memcached needs protection. The main limiting factor is this 1-second control loop and the overhead of changing Docker cpusets and memcached taskset assignments; below 1 second the load could change faster than the controller can reliably observe and react.

Use of AI Tools: Did you use any AI tools for this project? If so, disclose them here and explain what you used the tool(s) for and why. Note that the use of AI tools is NOT needed and NOT expected for the project.

We used AI tools as programming assistance during the project. In particular, AI helped us draft and refine scripts to automate the experimental pipeline, including cluster setup commands, experiment orchestration, result parsing, plotting, and packaging. AI was also used to support parts of the controller implementation and debugging, such as improving robustness, and identifying issues in log parsing or time alignment.